

Spiegazione esercizi Lezione 1 Python

```
1 # [[Collections#Le liste|Lists]]
```

Ripartendo dalla funzione `.append` che aggiungeva un elemento in ultima posizione all'interno della lista, torniamo a parlare delle variabili.

Le Variabili

Le variabili quando si dichiarano su python, gli si deve sempre mettere il tipo di variabile.

Es:



Python

```
1 list_1:list = [1,2,3,4,5]
2 a:str = "Ciao"
3 b:int = 5
```

Ritornando alle Liste, per crearne una bisogna dichiarare una variabile>assegnarle il tipo `list` > assegnarle dei valori nelle parentesi quadre (`[]`):



Python

```
1 list_1:list =[1,2,3,4,5]
```

⚠ Indicizzazione degli elementi nelle liste

Gli indici di una lista sono zero-based, cioè il primo elemento sta all'indice 0, il secondo sta all'indice 1 e così via

Le funzioni di python

La funzione `append`

Come detto sui gli altri appunti, questa funzione permette di aggiungere un elemento in fondo alla lista, per fare un esempio:



Python

```
1 list_1:list = [1,2,3,4,5]
2 list_1.append(6)
3 print(list_1)
```

In questo caso ho richiamato la lista e ci ho aggiunto l'elemento 6 in fondo (che guarda caso è un altro integer).

È possibile inserire una o più stringhe all'interno di questa lista composta da integer:



Python

```
1 list_1:list = [1,2,3,4,5]
2 list_1.append(6)
3 list_1.append("Hello, World!")
4 print(list_1)
```

Ma posso inserire liste all'interno di liste, creando delle nidificazioni di liste:



Python

```
1 # Una matrice 2x2
2 matrice: list = [[2, 1], [5, 3]]
3
4 # Un'immagine in bianco e nero (0 = bianco, 1 = nero)
5 img_bw: list = [[0, 1, 1], [1, 0, 0], [0, 0, 0]]
6
7 # Un'immagine in scala di grigi (valori float tra 0 e 1)
8 img_gray: list = [[0.0, 1.0, 1.0], [1.0, 0.5, 0.2], [0.1, 0.12, 0.3]]
9
10 # Un'immagine RGB (valori per ogni canale tra 0 e 255)
11 R: list = [[24, 128, 1], [10, 23, 225], [1, 0, 34]], # R
12 G: list = [[33, 27, 233], [5, 22, 245], [1, 2, 56]], # G
13 B: list = [[22, 128, 233], [10, 23, 255], [1, 0, 34]] # B
14
15
```

Come si vede da questo esempio le liste vengono usate anche per le immagini, che sono delle matrici questo perché le liste di liste sono matrici.

Ovviamente la variabile `img_bw`, come le altre variabili se la stampo non mi stampa anche l'immagine perché, ovviamente i valori vanno interpretati ma, a

livello concettuale, ci è utile a capire una cosa:

riprendendo l'esempio di `img_bw:list`, possiamo notare che questi sono i valori di un'immagine in bianco e nero; dove 0 = bianco e 1 = nero.

Nell'esempio subito sottostante (`img_gray`) i valori in float stanno ad indicare un'immagine in scala di grigi e le sotto-liste fanno riferimento ai valori RGB (vanno da 0 a 255).

Sovrascrivere un elemento delle liste partendo dal suo indice

Dichiariamo una lista:



Python

```
1 myList : list=[1,2,3,4,5,6]
```

Dopodiché voglio sostituire, in questa lista, l'elemento che sta all'indice 0 con una stringa:



Python

```
1 myList[0]="Ciao"
```

Quindi:



Python

```
1 myList : list=[1,2,3,4,5,6]
2 myList[0]="Ciao"
3 print(myList)
```

Mentre se avessi usato la funzione `.append` mi avrebbe solo aggiunto l'elemento in fondo alla lista all'indice indicatogli :



Python

```
1 R:list = [[24, 128, 1], [10, 23, 225], [1, 0, 34]], # R
2 G:list = [[33, 27, 233], [5, 22, 245], [1, 2, 56]], # G
3 B:list = [[22, 128, 233], [10, 23, 255], [1, 0, 34]]
4 B[0].append ("Ciao")
5 print(B[0])
```

La funzione di `len()`

È una funzione che mi restituisce la lunghezza dell'elemento messo come argomento.

Questa funzione è molto utile perché restituisce il numero di elementi contenuti in un oggetto, quindi viene usata quando si lavora con strutture di dati come liste, stringhe, tuple, dizionari e insiemi.

1. Casi d'uso base

1. Liste:



Python

```
1 my_list = [1, 2, 3, 4]
2 print(len(my_list)) # Output: 4
```

restituisce il numero di elementi nella lista

2. Stringhe:



Python

```
1 my_string = "Python"
2 print(len(my_string)) # Output: 6
```

Mi restituisce il numero di caratteri nella stringa, inclusi spazi e simboli

3. Tuple:



Python

```
1 my_tuple = (1, 2, 3)
2 print(len(my_tuple)) # Output: 3
```

Restituisce il numero di elementi nel tuple

4. Dizionari:



Python

```
1 my_dict = {"a": 1, "b": 2, "c": 3}
2 print(len(my_dict)) # Output: 3
```

Restituisce il numero di coppie chiave-valore

5. Insiemi o set:



Python

```
1 my_set = {1, 2, 3, 4}
2 print(len(my_set)) # Output: 4
```

Conta gli elementi unici nell'insieme.

2. Nelle liste annidate

La funzione `len()` trova la sua praticità anche nelle liste annidate, poiché restituisce la lunghezza del livello corrente:



Python

```
1 myNested_list = [[1, 2], [3, 4, 5], [6]]
2 print(len(myNested_list)) # Output: 3 (numero di sotto-liste)
3 print(len(myNested_list[1])) # Output: 3 (numero di elementi
  nella seconda sotto-lista)
```

Ritornando all'esempio delle variabili "R", "G", "B"; io adesso voglio sapere quanti elementi sono contenuti nella lista B:



Python

```
1 R:list = [[24, 128, 1], [10, 23, 225], [1, 0, 34]],
2 G:list = [[33, 27, 233], [5, 22, 245], [1, 2, 56]],
3 B:list = [[22, 128, 233], [10, 23, 255], [1, 0, 34]]
4 print(len(B))
5 print(len(B[0])) #su questa riga sto chiedendo di restituire la
  lunghezza del primo elemento della lista B
```

Tips & tricks

Dichiariamo 2 liste:



Python

```
1 g:list = [7,8,9]
2 f:list = [10,11,12]
3 g.append(f)
```

```
4 print(g)
5 print(g[3][0])
6 print(g[3][1:len(g[3])])
```

nel primo print stampo l'output che sarà

```
1 [7, 8, 9, [10, 11, 12]]
```

Questo conferma che la lista `f` è stata aggiunta come un unico elemento alla fine di `g`.

Con il secondo print: `print(g[3][0])`:

voglio stampare, tramite la prima parentesi quadra, il quarto elemento di `g`; cioè `[10, 11, 12]`.

Con la seconda parentesi quadra, accedo al primo elemento della lista `[10, 11, 12]`, che è `10`

Con l'ultimo print `print(g[3][1:len(g[3])])`, grazie allo slicing consente di selezionare un **sottoinsieme di elementi**:

- `g[3]` accede alla lista `[10, 11, 12]`.
- `[1:len(g[3])]` indica di prendere tutti gli elementi della lista a partire dall'indice `1` fino alla fine della lista (escluso l'ultimo indice).
- In `[10, 11, 12]`, l'indice `1` corrisponde al numero `11`.
- Il risultato dello slicing sarà `[11, 12]`.

Le funzioni di `.extend()` e `append()`

Le differenze sono:

1. `.extend()`:

posso aggiungere più elementi insieme nella lista



Python

```
1 a:list = [1,2]
2 b:list = [3,4]
3 a.extend(b)
4 print(a)
```

2. `append()`:

posso aggiungere un solo elemento in fondo alla lista



Python

```
1
2 a:list = [1,2]
3 b:list = [3,4]
4 a.append(b)
5 print(a)
```

Tip & Tricks

Volendo posso formattare l'output in questo modo,



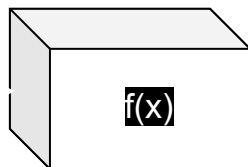
Python

```
1 print(f"funzione extend: {a+b}")
2 print(f"funzione Append: {a}")
```

Cosa sono le funzioni?

`.append()` e `.extend()` sono funzioni, una funzione può essere immaginata come una scatola:

Input(es: variabile a)



Output(non obbligatorio)

Diagram 1.svg

Come ci mostra questa immagine;

1. **Input :**

Riceve uno o più valori in ingresso (parametri o argomenti, in questo caso una variabile chiamata "a").

2. **Processo:**

Esegue un'operazione o una serie di operazioni basate sugli input.

3. **Output:**

Può restituire un valore (ma non è obbligatorio).

In matematica, una funzione si rappresenta come $f(x)$:

l'input è x e $f(x)$ è l'output).

Quindi, in questo senso, quello che fa la funzione `.append()` :
arriva l'input, elabora il dato che gli è arrivato all'interno della scatola, ma non restituisce nulla in uscita.

Difatti se scrivessi



Python

```
1 a:list = [1,2]
2 b:list = [3,4]
3 print(a.append(b))
```

mi darebbe `none` sul terminale, tuttavia modifica direttamente la lista: io gli dico di prendere come ingresso la variabile "a", quindi la lista "a", e come argomento gli passo la lista b, così facendo appende la lista b all'ultima posizione.

Sempre in questo senso la funzione `.extend()` aggiunge ogni elemento di un'altra iterabile (come una lista, una stringa o una tupla) alla lista originale e la modifica §



Python

```
1 a:list = [1,2]
2 b:list = [3,4]
3 a.extend(b)
4 print(a)
```

La proprietà `.real`

Restituisce solo la parte reale di un [Numeri Complessi](#).

Esempio Pratico



Python

```
1 z = 3 + 4j
2 print(z.real)
```

Output



Python

```
1 3.0
```

In python i numeri complessi sono rappresentati utilizzando la sintassi `a + bj` :

- `a` :
è la parte reale
- `b` :
è la parte immaginaria
Di conseguenza questa proprietà restituisce la parte reale del numero complesso.

Caratteristiche della proprietà `.real`

1. Restituisce un valore di tipo float
2. È una proprietà di sola lettura, quindi non può essere modificata direttamente

Nota

Se usata con un numero reale (`.real`), restituisce semplicemente il numero stesso



Python

```
1 x = 5
2 print(x.real)
```

La proprietà `.imag`

Restituisce la parte immaginaria di un numero complesso.



Python

```
1 z = 3 + 4j
2 print(z.imag) #output: 4.0
```

Quando hai un numero complesso nella forma `a + bj` , la proprietà `.imag` restituisce il valore della **parte immaginaria** `b` .



Python

```
1 # Numero complesso con parte immaginaria negativa
2 z2 = 5 - 2j
3 print(z2.imag) # Output: -2.0
4
5 # Numero reale (parte immaginaria è zero)
```

```
6 z3 = 7
7 print(z3.imag) # Output: 0.0
8
```

Dettagli importanti:

- La proprietà `.imag` restituisce sempre un **numero float**.
- È **di sola lettura**, quindi non può essere modificata direttamente.
- Se il numero è un numero reale, `.imag` restituirà `0.0`.

☰ Per ricapitolare

- `.extend()` aggiunge i singoli elementi dell'iterabile
- `.append()` aggiunge l'intero oggetto come unico elemento

Concatenazione con gli operatori

Concatenazione con l'operatore `+`

L'operatore `+` consente di concatenare due liste creando una **nuova lista**. A differenza di `append` ed `extend`, non modifica gli oggetti originali.



Python

```
1 a = [1, 2, 3]
2 b = [4, 5]
3
4 c = a + b # Crea una nuova lista unendo a e b
5 print(c) # Output: [1, 2, 3, 4, 5]
6 print(a) # Output: [1, 2, 3] (lista originale invariata)
```

In questo caso prende `a`, `b` e restituisce in output `a+b`.

Nota

Con `+`, il risultato è una nuova lista, e le liste originali rimangono immutate.

Prendiamo un altro esempio:



Python

```
1 h = a
2
3 h= 6
4
5 print(a)
6
7 print(h)
```

Concetto di assegnazione e mutabilità in Python

In Python, le variabili non "contengono" direttamente i valori, ma fungono da riferimenti (o "puntatori") a oggetti in memoria. Questo significa che assegnare una variabile ad un'altra non copia il valore, ma crea un nuovo riferimento allo stesso oggetto.

Vediamo nel dettaglio cosa accade nel codice.

Passaggio 1: Creazione della lista `a` e assegnazione a `h`

Creiamo una lista



Python

```
1 a = [1,2,3] # Crea una lista in memoria
2 h= a # 'h' punta alla stessa lista a cui punta 'a'
```

nella prima riga ho creato una lista assegnandola alla variabile `a` dopodiché la variabile `h` lo assegnata alla lista "a", a questo punto sia `a` che `h` puntano alla stessa area di memoria dove è memorizzata la lista `[1, 2, 3]`.

Quindi qualsiasi modifica fatta su questa lista tramite `a` o `h` sarà visibile da entrambe, perché condividono lo stesso riferimento.

Es:



Python

```
1 a = [1,2,3]
2 h=a
3 a[0] = 10
4 print(h) # Output: [10, 2, 3]
```

Modificando il primo elemento di `a`, anche `h` mostra la modifica, perché entrambe puntano alla stessa lista.

Passaggio 2: Cambiamento del valore di `h`



Python

```
1 a=[1,2,3]
2 h=a
3 h=6 # `h` ora punta a un nuovo oggetto, un intero con valore 6
```

Quindi ora ho assegnato un nuovo valore ad `h`, ma non ho modificato la lista originale.

Sto semplicemente dicendo che `h` ora deve puntare a un nuovo oggetto (un numero intero `6`).

🔗 L'assegnazione non influenza `a`

L'assegnazione non influenza `a`, che continua a puntare alla lista originale.



Python

```
1 print(a) # Output: [10, 2, 3]
2 print(h) # Output: 6
```

Viceversa se noi assegniamo un nuovo valore a `a` questo punterà al il nuovo valore assegnato mentre il valore di `h` rimane invariato



Python

```
1 a=[1,2,3]
2 h=a
3 a=6
4 print(a) #output: 6
5 print(h) #output: [1,2,3]
```

Ci si potrebbe chiedere se questa idea del puntatore renda difficile tenere traccia delle operazioni aritmetiche in Python, ma Python è impostato in modo che questo non sia un problema. Numeri, stringhe e altre *simple types* sono immutabili: **non è possibile modificarne il valore, ma è possibile modificare solo i valori a cui puntano le variabili**.

Per questo è sicuro fare operazioni del genere:



Python

```
1 x = 10
2 y = x
3 x += 5 #aggiunge 5 al valore di x e lo assegna a x
4 print ("x =", x)
5 print ("y =", y)
```



Python

```
1 x = 15
2 y = 10
```

Quando chiamiamo `x += 5`, non stiamo modificando il valore dell'oggetto `10` puntato da `x`; Stiamo piuttosto modificando la variabile `x` in modo che punti a un nuovo oggetto intero con valore `15`. Per questo motivo, il valore di `y` non è influenzato dall'operazione.

Passaggio 3: Creazione di copie

Se vuoi creare una **copia indipendente** della lista, puoi utilizzare vari metodi, come lo slicing o la funzione `list()`.

Es:



Python

```
1 a = [1, 2, 3]
2 h = a[:] # Copia completa della lista
3 h[0] = 10
4 print(a) # Output: [1, 2, 3] (la lista originale non cambia)
5 print(h) # Output: [10, 2, 3]
```

Con `h = a[:]`, crei una **nuova lista** in memoria, indipendente da `a`. Modificare `h` non avrà effetto su `a`.

Errore nell'indicizzazione

Il commento nel codice sottolinea un errore comune:



Python

```
1 a = 6
2 print(a[0]) # Questo darà un errore!
```

- Qui hai assegnato un numero intero a `a` (ad esempio, `6`).
- Gli interi non sono **iterabili** in Python, quindi non puoi accedere ai loro "elementi" con un indice come `[0]`.



Python

```
1 TypeError: 'int' object is not subscriptable
```

☰ Per Ricapitolare

- **Mutabilità:**
 - Le liste sono **mutabili**, quindi le modifiche apportate tramite un riferimento influenzano tutti i riferimenti.
 - Gli interi, invece, sono **immutabili**: assegnare un nuovo valore crea sempre un nuovo oggetto.
- **Riferimenti in memoria:**
 - Assegnare una lista a un'altra variabile non la copia, ma entrambe le variabili puntano alla stessa area di memoria.
 - Se vuoi una copia indipendente, usa lo slicing (`[:]`) o `list()`.
- **Indicizzazione:**
 - Solo le strutture dati **iterabili** (liste, stringhe, tuple, ecc.) possono essere indicizzate.
 - Gli interi non supportano l'indicizzazione.
- **Modifica del riferimento:**
 - Quando assegni un nuovo valore a una variabile, cambi solo il riferimento. Altri riferimenti all'oggetto originale non sono influenzati.

Qui a seguire l'esempio completo



Python

```
1 # Creazione di una lista
2 a = [1, 2, 3]
3 h = a # 'h' punta alla stessa lista di 'a'
4
5 # Modifica tramite 'h'
6 h[0] = 10
7 print(a) # Output: [10, 2, 3]
8 print(h) # Output: [10, 2, 3]
```

```
9
10 # Riassegnazione di `h`
11 h = 6 # Ora `h` punta a un nuovo oggetto (un intero)
12 print(a) # Output: [10, 2, 3] (rimane invariata)
13 print(h) # Output: 6
14
15 # Creazione di una copia della lista
16 h = a[:] # Crea una copia indipendente
17 h[1] = 20
18 print(a) # Output: [10, 2, 3] (nessun effetto su `a`)
19 print(h) # Output: [10, 20, 3]
```